Step 3: System and Object-Oriented Design

IMessageSerializer (Interface)

Purpose: The IMessageSerializer interface defines how messages are serialized and deserialized before being transmitted over the network.

Encapsulation: Message formatting logic is encapsulated within the interface, preventing other parts of the system from depending on specific message formats.

Single Responsibility: This interface is responsible only for message conversion and does not handle networking or business logic.

Controlled Use of Inheritance: The interface allows different serialization strategies to be implemented without changing existing server or client code.

ServerProcess (Abstract Class)

Purpose: ServerProcess encapsulates functionality common to all servers, including socket setup, connection handling, threading, and server lifecycle management.

Encapsulation: Internal details such as the server socket, thread pool, and running state are hidden within the class. External components interact with the server only through public methods like start() and stop().

Single Responsibility: This class is responsible only for managing server execution and networking, not for application-specific request processing.

Controlled Use of Inheritance: ServerProcess is abstract and cannot be instantiated directly. It defines shared behavior while delegating request handling to subclasses.

PrimaryServer (Extends ServerProcess)

Purpose: The PrimaryServer class represents the active server that processes client requests.

Encapsulation: Request-processing logic is encapsulated within the handleRequest() method, keeping it separate from networking and threading concerns.

Single Responsibility: This class focuses solely on handling client requests when acting as the primary server.

Controlled Use of Inheritance: It inherits common server behavior from ServerProcess and overrides only the parts that differ.

BackupServer (Extends ServerProcess)

Purpose: BackupServer represents a standby server that does not process client requests unless promoted.

Encapsulation: Promotion logic and backup-specific behavior are contained within the class, preventing misuse by external components.

Single Responsibility: The class is responsible for remaining on standby and transitioning to primary status when instructed.

Controlled Use of Inheritance: It reuses networking and concurrency logic from ServerProcess while implementing backup-specific behavior.

ConnectionHandler (Implements Runnable)

Purpose: ConnectionHandler manages a single client or server connection.

Encapsulation: Socket input/output handling is fully encapsulated within this class, preventing other components from dealing directly with low-level I/O.

Single Responsibility: Each instance handles exactly one connection, making the class simple and easy to reason about.

Controlled Use of Inheritance: The class implements Runnable, allowing it to be executed by a thread pool without inheriting unnecessary behavior.

HeartbeatSender (Implements Runnable)

Purpose: HeartbeatSender periodically sends heartbeat signals to indicate that a server is alive.

Encapsulation: Heartbeat timing and sending logic are hidden inside the class and not exposed to other components.

Single Responsibility: The class is responsible only for sending heartbeat messages at fixed intervals.

Controlled Use of Inheritance: By implementing Runnable, the class integrates with Java's concurrency model without extending unrelated classes.

FailoverManager

Purpose: The FailoverManager decides which backup server becomes the new primary when a failure occurs.

Encapsulation: Failover decision logic is contained within this class, preventing other parts of the system from directly manipulating server roles.

Single Responsibility: This class focuses exclusively on leader selection and does not handle networking or monitoring.

Controlled Use of Inheritance: No inheritance is used here.